# Campus Artisan Marketplace — Build Requirements (v1)

## Context for whoever is implementing this (including Claude Code)

This is a **two-sided marketplace** connecting student artisans (sellers of handmade/artisan products — jewelry, decor, textiles, pottery, etc.) with campus buyers. It launches campus-first, one college, with intent to expand to other colleges later.

**Business model:** commission-on-sale. The platform takes a percentage of each transaction; the rest settles to the artisan.

**Founding stage:** This is an unregistered/individual venture at this point — no LLP, Pvt Ltd, or GST yet. The founders (a student team) plan to formally register the business once it grows past pilot scale. Nothing in this build should assume a registered business entity exists. Payment collection is structured around **Razorpay's Individual/Unregistered Business KYC tier**, which is fully legal in India for this purpose (PAN + government ID + personal savings bank account — no GST/incorporation required).

**The core risk this build must respect:** the biggest competitive threat isn't other marketplaces — it's free substitutes (Instagram DMs, WhatsApp groups, college fest stalls) that artisans and buyers already use at zero cost. The platform only has a reason to exist if it's clearly better than those. Concretely, that means:

Friction for both sellers and buyers must be low (minimal signup, simple listing flow)
**Payment must happen ON the platform, not off it.** If checkout defers to "DM the seller to arrange UPI," the platform captures discovery but not the transaction — no commission, no real signal that anyone will pay through it, and no actual moat over Instagram. This is why real payment integration (not COD, not a static QR code) is a first-tier requirement, not a "V1 later" item, despite being the most common thing MVPs defer.
Everything else non-essential to proving that loop (seller dashboards, search/filters, reviews, notifications, multi-shop tooling, etc.) should be deferred past this build.

**Scale target for this build:** a single campus, realistically dozens of listings and low hundreds of users at most. Do not over-engineer for scale — optimize for speed of shipping and clarity of the core loop instead.

## 1. User Roles

| Role | Capabilities |
|---|---|
| **Buyer** | Browse listings, view product detail, checkout via Razorpay, view own order status |
| **Seller (artisan)** | Submit listing via a simple form (self-service, no dashboard needed yet) |

| Role | Capabilities |
|---|---|
| **Admin (founders)** | Approve/reject listings, view all orders, manually settle payouts to artisans, mark orders fulfilled |

Keep seller self-service minimal — a submission form is enough. Full seller dashboards (edit/delete own listings, sales stats) are a post-pilot addition once there are enough sellers that admin-managed listings become a bottleneck.

## 2. Listings

Fields: product name, category (jewelry / decor / textiles / pottery / other), price (INR), description, 1–3 photos, artisan name, artisan contact (phone/email/Instagram handle)
Submission: a coded "Add Listing" form, not a Google Form — this needs to write into the same database the storefront reads from
Display: grid view, newest-first. No search bar needed at this scale (dozens of items) — a category filter is sufficient
Status: Active / Sold — admin toggles manually after a sale, or automatically once an order is marked paid (see Orders section)

**Not in this build:** stock/quantity tracking, variants (size/color), image galleries beyond 3 photos, seller-facing edit access. Add these once listing volume makes manual admin management painful.

## 3. Discovery / Browsing

Single grid/feed, newest-first
Category filter (simple dropdown or tabs) — enough for dozens of listings
Product detail page: photos, description, price, artisan info, "Buy Now" button

**Not in this build:** full-text search, personalization/recommendations, wishlists, "trending" sections. None of these matter at campus-pilot listing volumes.

## 4. Orders & Payment — the core loop

This is the most important module in the build. It's where the platform either proves it can beat free substitutes or doesn't.

**Flow:**

Buyer clicks "Buy Now" on a product → sees order summary (item, price, platform's commission is invisible to buyer, baked into price or added transparently — decide once, be consistent) → enters name, phone, delivery/pickup preference
Buyer is taken to **Razorpay Checkout** (Payment Button or Payment Links / Orders API — see Technical Notes below) to pay the full listing price
On successful payment, Razorpay confirms via webhook/redirect → order status becomes "Paid" → listing auto-marks as "Sold" (or admin does this manually if webhook handling is deferred)
Admin sees the new paid order in an admin order list, with buyer contact + artisan contact
Admin manually transfers the artisan's share (price minus commission) to the artisan via UPI/bank transfer — **this payout step is manual for this build.** Automated split-payouts (Razorpay Route) require business-tier KYC per seller and are explicitly out of scope until the business is registered.
Order status: Placed → Paid → Fulfilled (admin marks Fulfilled once the artisan hands off / delivers the item)

**No COD, no external QR code, no "DM to arrange payment."** Every completed sale must go through Razorpay checkout — this is the one thing this build should not compromise on, since it's the whole point of testing this over Instagram/WhatsApp.

# 5. Payment Integration — Technical Notes for Implementation

**Account setup (founders do this before Claude Code needs live keys):**

Sign up at razorpay.com under **Individual / Unregistered Business** KYC category
Required: PAN card, government photo ID (Aadhaar/Voter ID/Passport), personal savings bank account details, of whichever founder's identity the account is registered under
No GST, no certificate of incorporation, no business current account needed at this tier
Note for the founders (not a build task): money collected legally lands as that individual's personal income for tax purposes until the business is formally registered — worth tracking separately from day one so it's not a mess later

**Integration approach (pick the lighter option first):**

**Simplest:** Razorpay **Payment Links API** — generate a unique payment link per order server-side, redirect buyer to it, Razorpay hosts the checkout page entirely. Minimal frontend work, webhook gives you payment confirmation.
**Slightly richer UX:** Razorpay **Checkout (Orders API + Checkout.js)** — an embedded/popup checkout on your own site rather than a redirect. More integration work, more polished feel.
Use test mode keys during development; switch to live keys only once KYC is approved.
Handle the `payment.captured` webhook (or equivalent) to reliably mark orders as paid — don't rely solely on client-side redirect success, since that can be spoofed or interrupted.

**Explicitly out of scope for this build:**

Razorpay Route / linked accounts / auto-split payouts (needs per-seller business KYC)

Recurring payments/subscriptions
International payment methods
Refund automation (handle refunds manually via Razorpay dashboard if needed at this scale)

## 6. Messaging / Communication

No in-app chat needed. Buyer and artisan contact details are exchanged via the order record (visible to admin, forwarded manually if needed)
Order confirmation to buyer: a simple confirmation screen/email after payment is enough; no need for a notification system

## 7. Admin Panel

A minimal internal-only view (can be auth-gated with a single shared admin password at this scale — no need for role-based auth system yet):

Pending listings queue → approve/reject
All orders list: buyer info, artisan info, item, amount, status (Placed/Paid/Fulfilled)
Manual "mark as Fulfilled" action
This data doubles as your validation dataset: listings submitted, orders placed, orders paid (conversion), repeat sellers/buyers — the numbers that tell you whether the platform is actually beating the free substitutes

## 8. Non-Functional Requirements

Mobile-responsive — non-negotiable, campus users are mobile-first
Fast load at small scale — no premature performance engineering needed
Buyers do not need accounts/login to purchase — every added step before checkout is friction that pushes people back to DM+UPI. Capture name/phone/email at checkout only.
Sellers submit via an unauthenticated or lightly-gated form (e.g., a shared code given only to onboarded artisans) rather than a full account system
Payment data itself is handled by Razorpay (PCI compliance is on them) — the app should never store card details; only store Razorpay order/payment IDs and status

Given: solo founder-developer, free-tier infrastructure, building with Claude Code.

**Frontend:** React (or plain HTML/CSS/JS if simpler) — storefront grid, product detail, listing submission form, admin panel
**Backend/DB:** Supabase or Firebase free tier — handles listings, orders, and admin data without standing up custom server infrastructure. Supabase is slightly better suited if webhook handling (a small server function) is needed for Razorpay payment confirmation
**Payments:** Razorpay (Payment Links API to start)
**Hosting:** Vercel/Netlify free tier for frontend; Supabase/Firebase functions for any backend logic (e.g., the Razorpay webhook handler)

## 10. What Success Looks Like (fill in before launch, not after)

Minimum # of artisans who list something in week 1: _____
Minimum # of buyer checkouts started in 2 weeks: _____
Minimum # of **completed, paid** orders in 2 weeks: _____
Conversion rate from "checkout started" → "paid" (tells you if Razorpay friction itself is a drop-off point)
Ask both sides directly: *"Why would/wouldn't you use this over Instagram/WhatsApp?"*

Since payment is now on-platform from day one, the paid-order count is a direct, honest measure of whether this beats the free substitutes — there's no manual-payment ambiguity clouding the signal anymore.